实验报告

数字逻辑与计算机组成实验

lab07：VGA 接口控制器实现

姓名：　法童

学号：　251250046

邮箱：　251250046@smail.nju.edu.cn

实验时间：　2026-10-16

1 实验目的

1. 理解 VGA 的行扫描与场扫描过程，掌握 640×480@60 的行、场时序参数与消隐时间。

2. 掌握 VGA 接口控制器的设计方法：用像素计数器同时产生行/场同步信号、消隐标志和当前像素坐标。

3. 用 Block RAM 实现显存，把”扫到哪个像素”与”这个像素显示什么”解耦，上层只改显存内容就能改变画面，不必关心扫描过程。

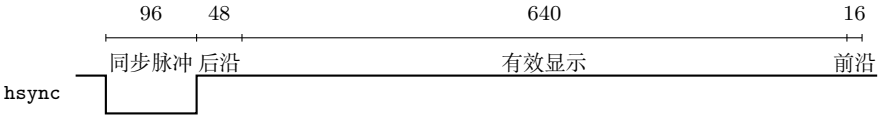
4. 拓展：自行完成图片到 coe 文件的转换，在显示器上显示自定义图片。

2 实验原理

2.1 VGA 的行、场时序

VGA 接口的模拟信号只有 5 根：R、G、B 三路颜色与 HS、VS 两个同步。板上用不同阻值的电阻把 FPGA 输出的数字信号合成为模拟电压，每种颜色给 4 位亮度信息，因此单色 16 级、三色组合 4096 种颜色。单色只有 16 级，颜色渐变的区域能看出分层，这是 4 位色深的固有限制。显示器逐行扫描，每扫完一帧从左上角重新开始，期间 RGB 不传送像素信息的时段称为行消隐和场消隐。

640×480@60 的一行由四段组成，其中只有第三段在传送像素：



一行共 $96 + 48 + 640 + 16 = 800$ 个像素时间，其中 160 个像素时间处于消隐；一帧共 $2 + 33 + 480 + 10 = 525$ 行，其中 45 行处于消隐。行、场同步都是**负脉冲**（低电平有效），脉冲宽度分别是 96 个像素时间和 2 行。

像素由 25 MHz 时钟驱动，一个时钟扫过一个像素点，因此一帧需要 $800 \times 525 = 420{,}000$ 个时钟周期，即 16.8 ms，对应的行频与场频为

$$f_H = \frac{25\,\text{MHz}}{800} = 31.25\,\text{kHz}, \qquad f_V = \frac{31.25\,\text{kHz}}{525} \approx 59.5\,\text{Hz}.$$

标准值是 31.469 kHz 与 59.94 Hz，偏差不到 1%，在显示器容差之内。

这里有一个容易踩的坑：**消隐期间必须把 RGB 置零**。CRT 的电子束在消隐期仍在扫描，如果这段时间 RGB 仍输出上一个有效像素的电压，亮度会被累加进相邻像素，画面看起来整体偏暗、发灰。所以颜色输出要由消隐标志选通。

2.2 显存的组织

一帧 640×480 个像素、每像素 12 位，共 3.69 Mbit，超过 Nexys A7-100T 上分布式 RAM 的 1.188 Mbit 上限，因此显存只能用 Block RAM（板上共 4.86 Mbit）。

显存单元的排布要和地址的计算方式一起考虑。若按行优先存放，地址为 $v_addr \times 640 + h_addr$，需要一次乘法；若按**列优先**存放、并给每列预留 512 行的空间，地址就变成

$$\text{addr} = \{h_addr,\ v_addr[8{:}0]\} = h_addr \times 512 + v_addr,$$

乘 512 就是把列号左移 9 位，**用拼接就能实现，不需要任何运算**。代价是每列 480 行真实数据后面要补 32 个空位，显存深度从 $640 \times 480 = 307{,}200$ 增加到 $640 \times 512 = 327{,}680$。

同时，行号只取低 9 位参与寻址，而 v_addr 的实际范围是 0∼479，永远到不了填充区，因此填充值取什么都可以，只要把 coe 文件凑够 327,680 项即可。

3 实验环境与器材

- 开发板：Nexys A7-100T（FPGA 型号 xc7a100tcsg324-1），板载 VGA 接口
- 开发工具：Vivado 2020.2（非工程模式，`build.bat` / `prog.bat` 脚本驱动）
- 硬件描述语言：Verilog HDL
- 输出设备：支持 640×480 的显示器与 VGA 线

4 设计思路

系统分三块：时基（`tickgen`）、扫描控制器（`vga_ctrl`）、显存（`blk_mem_gen_0`），由 `top` 接板级端口。

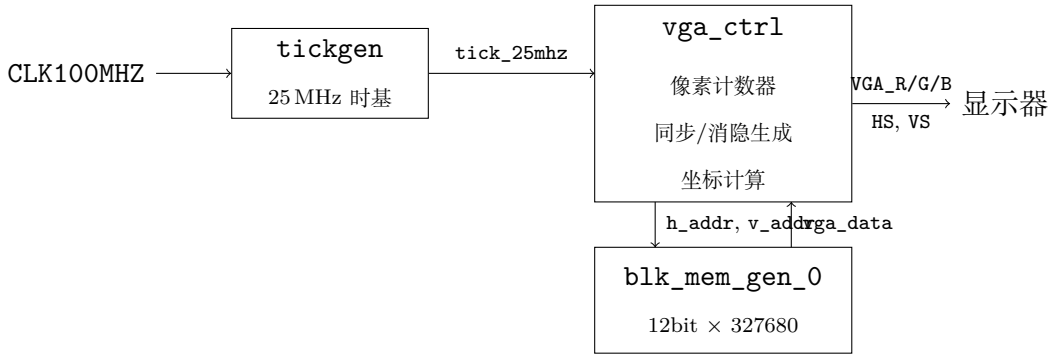


图 1: 系统框图

表 1: 板级资源分配

板级资源	顶层信号	功能
CLK100MHZ	CLK100MHZ	100 MHz 系统时钟
CPU_RESETN	reset	复位，取反后至高有效
VGA_R[3:0]、VGA_G[3:0]、VGA_B[3:0]	同名	三路各 4 位的颜色输出
VGA_HS、VGA_VS	同名	行、场同步（负脉冲）

4.1 扫描控制器

vga_ctrl 用两个计数器描述扫描位置：x_cnt 计一行内的像素，y_cnt 计帧内的行，两者都以 25 MHz 时基递增。由这两个计数值直接得到同步、消隐与坐标：

```
assign hsync = (x_cnt > h_frontporch);                    //低电平脉冲宽 96
assign vsync = (y_cnt > v_frontporch);                    //低电平脉冲宽 2 行
assign h_valid = (x_cnt > h_active) & (x_cnt <= h_backporch);
assign v_valid = (y_cnt > v_active) & (y_cnt <= v_backporch);
assign valid = h_valid & v_valid;
assign h_addr = h_valid ? (x_cnt - 10'd145) : {10{1'b0}};  //当前像素坐标
assign v_addr = v_valid ? (y_cnt - 10'd36)  : {10{1'b0}};
assign vga_r = valid ? vga_data[11:8] : 4'b0000;          //消隐期置零，避免发暗
assign vga_g = valid ? vga_data[7:4]  : 4'b0000;
assign vga_b = valid ? vga_data[3:0]  : 4'b0000;
```

其中 h_addr 取 0∼639、v_addr 取 0∼479，正好是当前像素在有效显示区内的坐标，交给上层去索引显存。25 MHz 时基沿用 lab04 的分频器，把 100 MHz 四分频得到：

```
tickgen #(.tick_freq(25000000)) u_tick_25mhz (
    .clk(CLK100MHZ), .rst(1'b0), .en(1'b1), .tick(tick_25mhz)
);
```

4.2 显存

显存用 blk_mem_gen 生成一块单口 Block RAM，宽 12 位、深 327,680，使能恒定，读出打一拍，并用 coe 文件初始化。这些配置写在 lab 目录下的 ip.tcl 里，由综合脚本自动调用：

```
set_property -dict [list \
    CONFIG.Memory_Type {Single_Port_RAM} \
    CONFIG.Write_Width_A {12} \
    CONFIG.Write_Depth_A {327680} \
    CONFIG.Enable_A {Always_Enabled} \
    CONFIG.Load_Init_File {true} \
    CONFIG.Coe_File [file normalize ./picture_2.coe] \
] [get_ips blk_mem_gen_0]
```

显存只读不写，因此写使能与写数据接常数，地址由两个坐标拼成：

```
blk_mem_gen_0 u_vram (
    .clka (CLK100MHZ),
    .wea  (1'b0),
    .addra({h_addr, v_addr[8:0]}),    //列号在高位 -> 列优先、步长 512
    .dina (12'd0),
    .douta(vga_data)
);
```

需要注意的是 Block RAM 是同步读，douta 比 addra 晚一个时钟。本设计里一个像素占 4 个主时钟，晚一拍只影响其中 1/4 的时间，而显示器是在像素中心采样，静态图片上看不出差别。

5 实验过程

1. 复用 lab04 的 tickgen，把 100 MHz 四分频得到 25 MHz 时基。
2. 按手册实现 vga_ctrl 的两个像素计数器与各输出信号；**颜色输出额外用消隐标志选通**，把手册提到的”画面发暗”问题一并解决。
3. 编写 ip.tcl，用 blk_mem_gen 生成 12bit × 327680 的单口 RAM，由课程提供的 picture_1.coe 初始化。
4. 编写 top.v，连接时基、vga_ctrl 与显存，地址按列优先拼接。
5. 从根目录 nexysa7.xdc 复制约束，取消 CLK100MHZ、CPU_RESETN、VGA_R/G/B、VGA_HS、VGA_VS 的注释。

6. 运行 `build.bat lab07\vga` 生成比特流，`prog.bat` 下载上板，接显示器观察默认图。

7. **拓展要求**：重写 `coe_generate.py`，完成图片到 coe 的转换，换上自己的图片重新生成显存并上板。

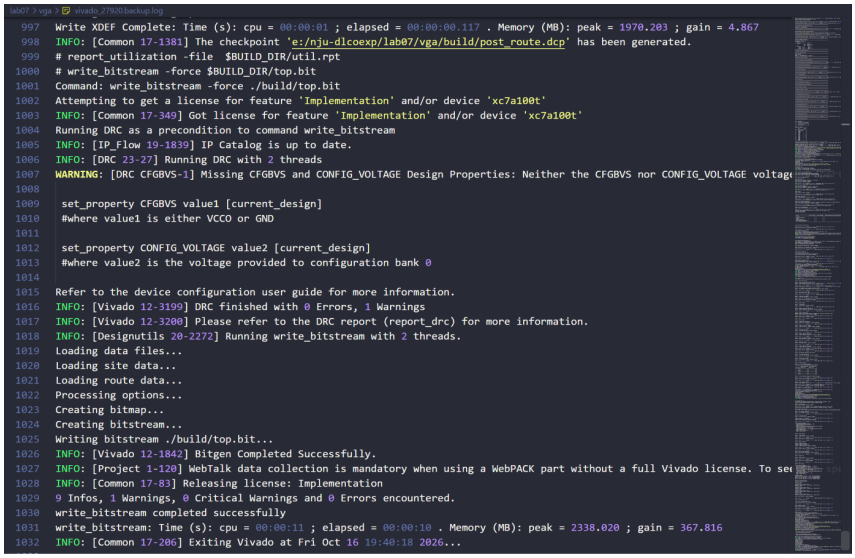


图 2: 综合、实现与比特流生成完成

6 测试方法

如何验证：本次验证以上板显示为主。
先用手册提供的默认图（北大楼，彩色、细节丰富、有明确的天空/建筑/地面层次）做上板验收，再换自定义图片验证拓展要求。

上板操作：接好 VGA 线后 `prog.bat` 下载，观察显示器；换自定义图时改 `ip.tcl` 中 `Coe_File` 的路径、重新 build 与 prog 即可。

7 实验结果

7.1 上板

默认图显示正常：蓝天、灰瓦屋顶、绿树与红色塔顶的位置和颜色与源图一致，画面方向、左右位置都没有偏移，说明像素计数、坐标计算和显存的地址排布三者对得上。



图 3: 上板实测：课程默认图（北大楼）正常显示

7.2 拓展要求：显示自定义图片

把手册的转换脚本重写为对任意图片都可用，流程是：居中裁剪成 4:3 → 缩放到 640×480 → 每通道取高 4 位 → 按列优先写入、每列补足 512 项。核心的两层循环与填充如下：

```
vals = []
for x in range(W):                    # 列优先：外层是列号
    vals += ["%X%X%X" % (pix[x, y][0] >> 4, pix[x, y][1] >> 4, pix[x,
        y][2] >> 4)
             for y in range(H)]
    vals += [FILL] * (STRIDE - H)     # 每列补到 512
```

脚本的正确性用课程提供的 coe 反向校验：把默认图的内容区经本脚本重新转换一遍，生成的 327,680 项与 `picture_1.coe` **逐项完全相同**，说明列优先的顺序、512 的步长、填充的位置和取高 4 位的取法都一致。把自己的图片按同一流程转换后替换显存初始化文件，上板显示正常。

7.3 综合后的 Schematic

综合后的层次与设计框图一致：`top` 下面是 `tickgen`、`vga_ctrl` 和 `blk_mem_gen_0`。显存是这次资源消耗的主体，单块 12bit × 327680 的 Block RAM 占掉 110 块 `RAMB36`，占全芯片 135 块的 81.5%；其余逻辑很轻，`vga_ctrl` 的两个计数器和组合逻辑一共只



图 4: 上板实测：自定义图片正常显示

用 365 个 LUT、65 个寄存器。

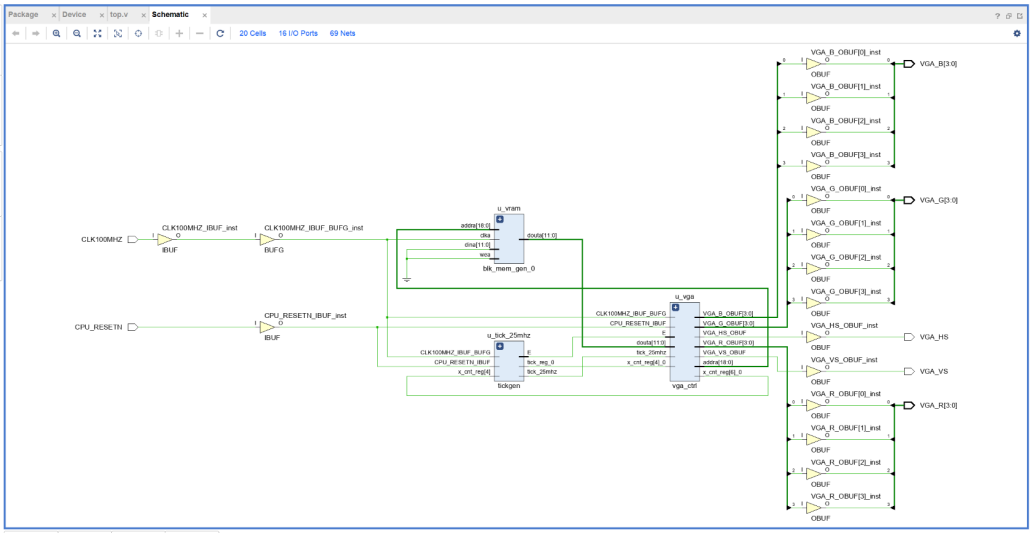


图 5: 综合后的 Schematic：顶层与三块的层次

8 问题与解决办法

- **按手册提供的转换脚本生成的 coe 不能用**：手册给的图片转 coe 代码按图片的高度（480）作为每列的步长来写，而显存的地址是 $\{h_addr, v_addr[8{:}0]\}$、步长必

须是 512，两者差 32 个填充项。直接拿它生成的 coe 初始化显存，显示出来的图会斜着错位。

解决：自己重写转换脚本，在每列 480 行真实数据之后补 32 个填充项凑足 512，并用课程提供的 `picture_1.coe` 反向校验转换结果。

- **上板后显示器报" 找不到 VGA 输入"**：程序下载成功、板上的 DONE 指示灯亮，说明配置没有问题，故障不在下载环节。确认设计本身无误，从而把问题定位到显示设备一侧。

  解决：更换显示器与 VGA 线后正常显示。

- **想做两张图片的切换，但显存放不下**：两张 640×480、12bit 的图共需 7.86 Mbit，而板上 Block RAM 一共只有 4.86 Mbit，一张就已经占掉 3.93 Mbit。让 Vivado 生成" 两张图前后排布" 的显存 IP（12bit × 655360），工具直接报错 `User configuration exceeds BRAM count in the selected device`；

  解决：放弃采用两张图片切换，沿用单张图片静态显示。

9 启示与建议

启示：

1. 显示控制器把" 扫到哪" 和" 显示什么" 分开了：`vga_ctrl` 只负责数出当前扫到第几行第几列，画面内容完全交给显存。这样换一张图只要换显存里的数据，成功把" 时序" 和" 内容" 解耦。

2. 动手加功能之前应该先算资源的账。我一开始想当然地认为" 再放一张图而已"，直到算了两倍的总量和片上 Block RAM 的容量，才发现根本装不下，而且降到 8 位色深仍然不够。

建议：手册在讲显存时提到了 "需要自行计算具体显示像素对应到存储图片的地址"，但没有说明课程 coe 文件本身的排布规则。而手册同时附带的转换代码恰恰是按另一种步长生成的，直接使用会得到斜掉的图。建议手册直接写明 coe 的存储格式，或者把附带的转换代码与显存地址对齐。